

COL331 Operating Systems

Copy on Write with Swapping in xv6

Harsha Vardhan Somisetty 2021CS10390

Medicharla Neeraj 2021CS10087

Kotikala Raghav 2021CS50616

Data Structures in `rmap`:

`rmap` is implemented as an array of `rmap_node` of size `(PHYSTOP >> 12)`.

`rmap_node` is defined in `cow.h`:

```
struct rmap_node {
    pde_t *pgdirs[NPROC];
};
```

We chose to store only the page directory address of the process in the `rmap_node` structure.

This decision was made considering the fact that `copyvm` and `initvm` do not have access to the child process.

As modifying these signatures may lead to inconsistencies, we have only stored the page directory address. Assuming that the page directory of a process does not usually change during its lifetime, we can safely assume a bijection between a process and its pgdir addr. Page directories are stored in an array instead of a linked list because they can potentially contain `NPROC` many elements. Hence we anyway had to maintain these many buffers. In the linked list design, there is also an overhead of maintaining a pointer per buffer, which would have effectively doubled the size of `rmap`. Hence, without any guarantees on the number of processes accessing a page, it's better to maintain an array here instead of a linked list.

Functions of `rmap`:

1. `void rmap_add(uint addr, pde_t *pgdir)`
Adds a pagedir address to `rmap_node` corresponding to `addr`. It replaces the first zero element in the array.
2. `void rmap_rm(uint addr, pde_t *pgdir)`
Removes a pagedir addr from the `rmap_node`'s `pgdirs` array and moves all the following elements in the array backward.
3. `uint rmap_getcnt(uint addr)`
Get the number of processes pointing to a physical page.
4. `void rmap_clean()`
Cleans the `rmap` at initialization.

This function is called at the end of `seginitt` in `vm.c`

5. `void rmap_pgflt(uint va, pde_t *pgdir)`

This is called when a page fault occurs, and the page is present but not writable. If only one process accesses this page, we mark the page table entry writable. Else, we allocate a new physical page for the process and make it writable.

The TLB cache is flushed after this by `lcr3` instruction.

All the above functions first acquire a spinlock before performing any operations on `rmap`. Spinlock is used instead of Sleeplock because we ensure less cpu time between locking and unlocking. So the overhead of sleep might be more than the loss due to spin.

Changes in `vm.c`:

- In `copyvm`, we call `rmap_add` to add the page directory address of the child to `rmap`.
- In `deallocvm`, we call `rmap_rm` and check whether the page is accessed by only one process. If the page is accessed by more than one process, we do not call `kfree`.
- Similarly, we call `rmap_add` in `allocvm` and `initvm`.

Modifications in `pageswap.c`:

- We extend the struct `swapslot` to also include a `rmap_node`. So the modified `swapslot` is:

```
struct swapslot{
    int page_perm;
    int is_free;
    struct rmap_node rmap_node;
};
```

- While checking accessed bits, we search all the page directories corresponding to the physical address in `rmap`. Only if all of them are present and not accessed we select the page a victim.
- We clear accessed bits from all the page directories in `rmap`.
- During swap out, the slot number is written on the page table entries of all the page directories, and all their TLB entries are flushed. We then search the process table to find the corresponding processes and decrease their `rss` values.
- During swap in, we restore the physical addresses and permissions in all the page tables. This is clearly necessary since all of them point to the same physical page. Again, we clear their TLB entries and increase all `rss` values after searching the process table.
- The swap in function manages all page faults. So we first check whether the page is already present and not writable. If so, we forward to `rmap_pgflt` and return.
- During clearing swap slots after a process exits, we first check how many processes access this page. If only one process accesses this page, we free the slot. Else we just remove the `pgedir` corresponding to the process.